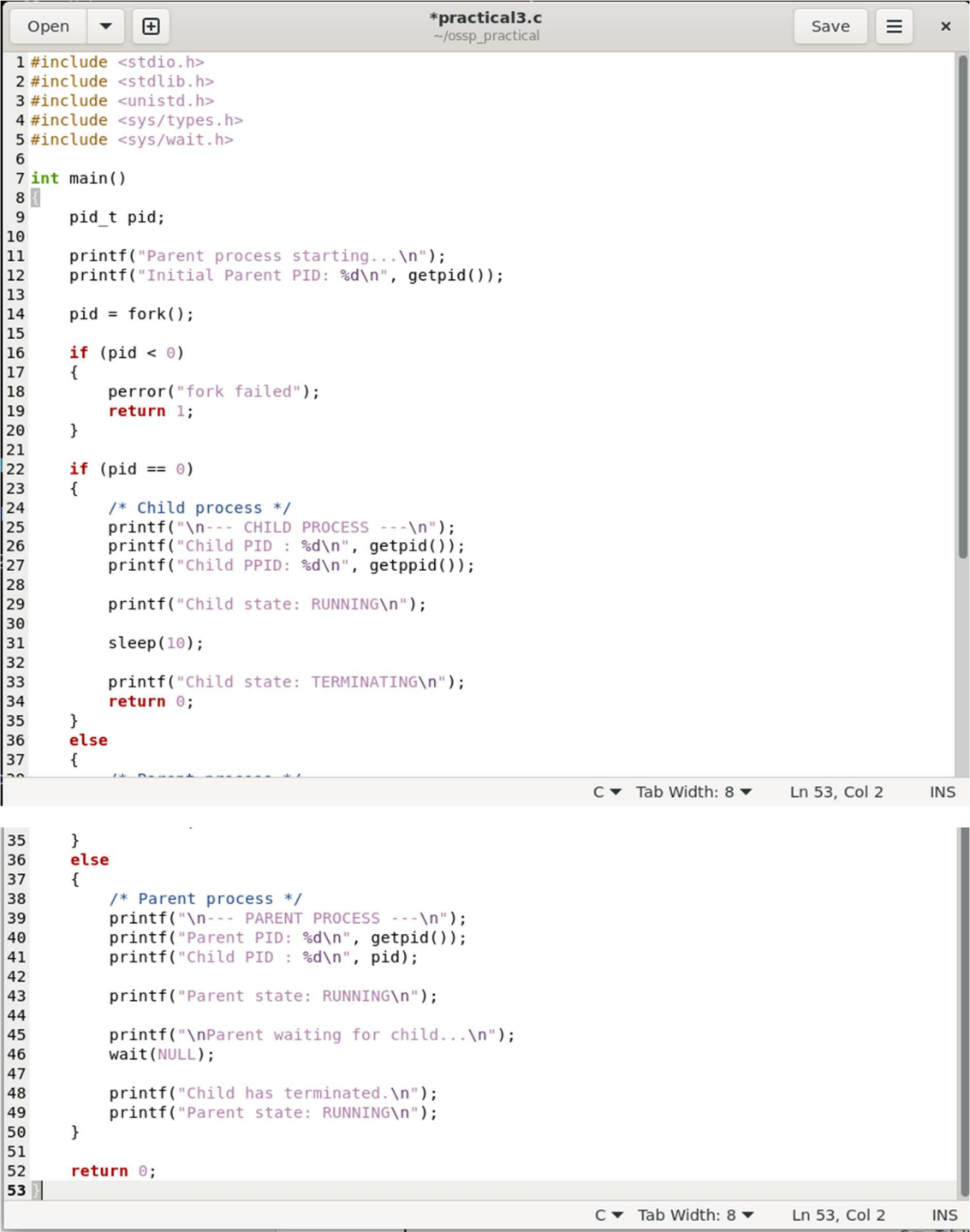


Practical – 3 – OS



The image shows a code editor window titled '*practical3.c' with the file path '~/.ossp_practical'. The code is written in C and demonstrates process creation and synchronization using `fork()` and `wait()`. The parent process starts, prints its PID, and then forks a child process. The child process prints its own PID and PPID, sleeps for 10 seconds, and then terminates. The parent process then waits for the child to finish and prints a message when it has terminated.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6
7 int main()
8 {
9     pid_t pid;
10
11     printf("Parent process starting...\n");
12     printf("Initial Parent PID: %d\n", getpid());
13
14     pid = fork();
15
16     if (pid < 0)
17     {
18         perror("fork failed");
19         return 1;
20     }
21
22     if (pid == 0)
23     {
24         /* Child process */
25         printf("\n--- CHILD PROCESS ---\n");
26         printf("Child PID : %d\n", getpid());
27         printf("Child PPID: %d\n", getppid());
28
29         printf("Child state: RUNNING\n");
30
31         sleep(10);
32
33         printf("Child state: TERMINATING\n");
34         return 0;
35     }
36     else
37     {
38         /* Parent process */
39         printf("\n--- PARENT PROCESS ---\n");
40         printf("Parent PID: %d\n", getpid());
41         printf("Child PID : %d\n", pid);
42
43         printf("Parent state: RUNNING\n");
44
45         printf("\nParent waiting for child...\n");
46         wait(NULL);
47
48         printf("Child has terminated.\n");
49         printf("Parent state: RUNNING\n");
50     }
51
52     return 0;
53 }
```

Practical – 3 – OS

```
siddharth@SiddharthReddy: ~  
siddharth@SiddharthReddy:~/ossp_practical$ gedit practical3.c  
siddharth@SiddharthReddy:~/ossp_practical$ cd ~/ossp_practical  
gcc practical3.c -o practical3  
siddharth@SiddharthReddy:~/ossp_practical$ cd ~/ossp_practical  
./practical3  
Parent process starting...  
Initial Parent PID: 1115  
  
--- PARENT PROCESS ---  
Parent PID: 1115  
Child PID : 1116  
Parent state: RUNNING  
  
Parent waiting for child...  
  
--- CHILD PROCESS ---  
Child PID : 1116  
Child PPID: 1115  
Child state: RUNNING  
|
```

```
siddharth@SiddharthReddy:~/ossp_practical$ ps -o pid,ppid,state,stat,cmd -p 1190  
PID PPID S STAT CMD  
1190 1189 S S+ ./practical3  
siddharth@SiddharthReddy:~/ossp_practical$ cat /proc/1190/status | grep -E 'Name|State|Pid|PPid|TracerPid:'  
Name: practical3  
State: S (sleeping)  
Pid: 1190  
PPid: 1189  
TracerPid: 0  
siddharth@SiddharthReddy:~/ossp_practical$ |
```

```
Parent state: RUNNING  
Parent waiting for child...  
  
--- CHILD PROCESS ---  
Child PID : 1176  
Child PPID: 1175  
Child state: RUNNING  
Child state: TERMINATING  
Child has terminated.  
Parent state: RUNNING  
siddharth@SiddharthReddy:~/ossp_practical$ ./practical3  
Parent process starting...  
Initial Parent PID: 1179  
  
--- PARENT PROCESS ---  
Parent PID: 1179  
Child PID : 1180  
Parent state: RUNNING  
  
Parent waiting for child...  
  
--- CHILD PROCESS ---  
Child PID : 1180  
Child PPID: 1179  
Child state: RUNNING  
Child state: TERMINATING  
Child has terminated.  
Parent state: RUNNING  
siddharth@SiddharthReddy:~/ossp_practical$ ./practical3  
Parent process starting...  
Initial Parent PID: 1189  
  
--- PARENT PROCESS ---  
Parent PID: 1189  
Child PID : 1190  
Parent state: RUNNING  
  
Parent waiting for child...  
  
--- CHILD PROCESS ---  
Child PID : 1190  
Child PPID: 1189  
Child state: RUNNING
```